

CSEN 296B-2 | Week 3.1 Lab Submission

Refined Requirement Baseline for AstraNotes

Student Name: Atishay Jain | Date: April 13, 2026 | Project: AstraNotes | Technical Path: Python

1. Refined Requirement Baseline

The requirements below are a direct refinement of the Week 1.2 Requirement Set. Each requirement has been reviewed for ambiguity, weak assumptions, and missing edge cases identified during the Week 2.1 and Week 2.2 planning work (Working Agreement, Definition of Done, user stories, backlog, and Sprint Zero plan). Changes are noted inline. Requirements that were already precise are carried forward with minor clarifications only.

1.1 Functional Requirements

ID	Refined Requirement	Change from Week 1.2
FR-01	The system shall allow a user to create a new text note with a non-empty title and a body (which may be empty). The note shall be assigned a UUID, created_at timestamp, and modified_at timestamp automatically at creation time. Titles consisting only of whitespace are treated as empty.	Added: title must be non-empty, enforced by ValidationLayer. Body may be empty. Timestamps are system-assigned, not user-provided. Whitespace-only titles are explicitly rejected.
FR-02	The system shall allow a user to edit the title or body of an existing note and save the updated version to local storage. On save, the modified_at timestamp shall be updated to the current time. The created_at timestamp and the note's UUID shall not change.	Added: explicit constraint that created_at and UUID are immutable on edit. Clarified that modified_at updates on every save, not only when content differs.
FR-03	The system shall allow a user to delete a note by its UUID. If the UUID does not exist in the collection, the system shall surface a clear error rather than silently succeed or crash.	Added: behavior for deleting a non-existent note. Original was silent on this edge case.
FR-04	The system shall allow a user to toggle a note's is_private flag. When is_private is True, the note body shall be encrypted by PrivacyService before being written to disk, and decrypted by PrivacyService on retrieval. Changing is_private from True to False shall re-save the note with a plaintext body. If decryption fails during the toggle, the operation shall fail with a clear error and the note shall remain encrypted.	Added: behavior for toggling privacy off. Added: failure handling when decryption fails during toggle. Original only described the encrypt-on-save direction.
FR-05	The system shall persist notes to local storage using a one-file-per-note format. On startup, all valid note files shall be loaded. Files that are missing, corrupt, or unreadable shall be skipped with an error logged; remaining notes shall load normally. If the data directory does not exist on first launch, the system shall create it.	Added: explicit handling for corrupt or missing files on startup (was all-or-nothing). Added: directory auto-creation on first launch. Separated the storage format decision (JSON) from the persistence requirement per class guidance on requirements vs. design.
FR-06	Each note shall carry metadata: a unique identifier (UUID), created_at (datetime, UTC), modified_at (datetime, UTC), and tags (list of strings, may be empty). Tags shall be stored as entered (case-sensitive).	Added: UTC requirement for timestamps. Clarified that tags are case-sensitive and may be an empty list.
FR-07	The system shall allow keyword search across note titles and bodies. Search shall be case-insensitive and shall use plain string matching (not regex). Private notes that are successfully decrypted shall be included in search results. Notes that cannot be decrypted shall be excluded with an error logged. An empty collection shall return an empty result set, not an error.	Added: explicit behavior for private notes in search. Added: plain string matching to avoid regex edge cases. Added: empty collection behavior. Original did not address any of these scenarios.

FR-08	The system shall display notes sorted by modified_at descending by default. Notes with identical modified_at values shall be ordered by created_at descending as a stable tiebreaker.	Added: stable tiebreaker for equal modified_at values. Original left sort order undefined for ties.
-------	---	---

1.2 Non-Functional Requirements

ID	Refined Requirement	Change from Week 1.2
NFR-01	The system shall complete note listing and keyword search within 2 seconds for a local collection of up to 500 notes on a machine meeting the minimum development hardware baseline (8 GB RAM, SSD storage, modern dual-core CPU).	Added: hardware baseline definition. Original stated the performance target without defining the measurement environment.
NFR-02	The system architecture shall enforce separation between the business logic layer (NoteManager), the storage layer (NoteRepository / JsonFileRepository), and the privacy layer (PrivacyService) such that each can be unit-tested independently without instantiating the others.	Added: testability as an explicit constraint. Named the layers using architecture component names from Week 1.2. Original was vague about what 'separation' required.
NFR-03	The application shall load and display the note list within 3 seconds of launch on a machine meeting the hardware baseline defined in NFR-01, measured from process start to first interactive display.	Added: measurement definition (process start to first interactive display). Referenced NFR-01 hardware baseline for consistency.

1.3 Security, Privacy, Reliability, and Governance Requirements

ID	Refined Requirement	Change from Week 1.2
SPR-01	Private notes (is_private == True) shall have their body encrypted using Fernet symmetric encryption (Python cryptography library) before any write to disk. The encryption key shall never be stored in plaintext in source code or in the notes data directory. Key management strategy shall be documented in the Architecture Decision Log before Sprint 1 begins.	Added: key storage constraint (must not be in source code or data directory). Added: requirement to document key management in ADL before Sprint 1. Original deferred key management without a deadline.
SPR-02	The system shall catch all storage-layer exceptions (FileNotFoundError, PermissionError, JSONDecodeError, and equivalent) and translate them into application-level error messages that do not expose file paths, stack traces, or internal state to the user.	Added: specific exception types. Added: constraint against exposing file paths and internal state. Original said 'non-technical error messages' without specifying what must be hidden.
SPR-03	All core components (NoteRepository, PrivacyService, ValidationLayer) shall have at least one automated unit test covering their primary behavior before each milestone delivery. Tests shall run without external dependencies (no real file system access, no real encryption key required; use temp dirs and test keys).	Added: tests must run without external dependencies (mocking / temp dirs). Original required tests but did not prevent tests that depend on real infrastructure.
SPR-04	Any third-party library added to the project shall be recorded in pyproject.toml or requirements.txt with a pinned version and a one-line comment explaining its purpose and confirming its license is permissive (MIT, Apache 2.0, BSD, or equivalent). Libraries without a confirmed permissive license shall not be added.	Added: pinned versions. Added: license confirmation requirement (must be permissive). Original said 'document purpose and license' without requiring pinning or validation.

2. Ambiguity Review

The following ambiguities were identified in the original Week 1.2 requirement set by reviewing each requirement against the user stories and acceptance criteria from Week 2.2. Each item describes the ambiguity, why it matters, and how it was resolved in the refined baseline above.

A-01 - FR-01 - What counts as a valid title?

Ambiguity: The original requirement said "a title and body" without specifying whether an empty string or a whitespace-only string is acceptable. This matters because the ValidationLayer behavior and the US-01 acceptance criteria depend on a clear rule for what gets rejected.

Resolution: Title must be non-empty after stripping whitespace. Body may be empty. This is enforced by ValidationLayer (raises ValidationError) and is now explicit in refined FR-01.

A-02 - FR-04 - What happens when privacy is toggled off?

Ambiguity: The original requirement described encrypting on save when is_private is True but said nothing about the reverse direction. This creates an undefined state: a note could have an encrypted body on disk but is_private set to False, making the stored content unreadable.

Resolution: Toggling is_private from True to False re-saves the note with a decrypted plaintext body. If decryption fails during the toggle, the operation fails with a clear error and the note remains encrypted.

A-03 - FR-07 - Are private notes included in search results?

Ambiguity: The original search requirement said "search notes by title or body content" without addressing private notes. If note bodies are encrypted on disk, searching them requires decryption first. The original was silent on whether search should attempt this, or whether private notes should be invisible to search.

Resolution: Private notes that decrypt successfully are included in search. Notes that cannot be decrypted are excluded and an error is logged. This is consistent with the layered architecture where PrivacyService sits between storage and business logic.

A-04 - NFR-01 and NFR-03 - On what hardware?

Ambiguity: Both performance requirements stated time targets (2 seconds for search, 3 seconds for startup) without defining the measurement environment. A low-specification machine could fail these targets even with a correct implementation, making the requirements untestable in practice.

Resolution: A hardware baseline is defined in NFR-01 (8 GB RAM, SSD, modern dual-core CPU) and referenced in NFR-03. This makes the performance targets reproducible and defensible.

A-05 - SPR-01 - Where is the encryption key stored?

Ambiguity: The original requirement required Fernet encryption but said nothing about where the key is stored. This is the most significant security gap in the original requirement set. A key stored in the notes data directory or hardcoded in source would make the encryption meaningless.

Resolution: The key must not be stored in source code or the notes directory. The specific mechanism (environment variable, OS keychain, user prompt at startup) is deferred, but a key management strategy must be documented in the ADL before Sprint 1 begins. This bounds the deferral.

A-06 - FR-05 - What does persistence mean versus how it is stored?

Ambiguity: The original requirement specified JSON as the storage format. The class guidance on requirements versus design decisions (Week 3.1 live examples) made clear that a storage format is a design choice, not a requirement. The actual requirement is that notes survive between sessions.

Resolution: Refined FR-05 states the persistence requirement (one-file-per-note, corrupt-file handling, directory auto-creation) without mandating JSON. The JSON File Adapter is documented as a design decision in the Architecture Decision Log, not as a requirement constraint.

3. Edge-Case Review

The following edge cases were identified by walking through each user story's acceptance criteria from Week 2.2 and asking: what could go wrong that the requirement does not currently address? Cases are grouped by the architecture component they affect.

Storage Layer (JsonFileRepository)

- Note file is corrupt or contains malformed JSON on startup

FR-05 now specifies that corrupt files are skipped with an error logged. The application must not crash or refuse to start because of a single bad file.

- **Note data directory does not exist on first launch**

JsonFileRepository must create the data directory if it does not exist. This was not addressed in the original requirements and is now an implementation constraint in FR-05.

- **Two notes saved in rapid succession share the same modified_at timestamp**

FR-08 now requires a stable tiebreaker (created_at descending). Without this, the sort order would be non-deterministic and the note list would shift unpredictably.

- **A note file exists on disk but the UUID in the filename does not match the UUID inside the JSON body**

The system should treat the UUID inside the file content as authoritative and log a warning about the mismatch. This prevents silent data corruption from manual file renaming.

PrivacyService

- **Decryption fails because the encryption key has changed since the note was encrypted**

PrivacyService must catch decryption failures and surface a clear error. The note must not be returned with garbled content. Addressed in refined FR-07 for search and applies to all retrieval operations.

- **A note is marked private but PrivacyService is unavailable or the key is missing**

The system must not silently return unencrypted content. It must raise an error and block the read. This is an implicit constraint of SPR-01 that is now explicit.

- **Toggling privacy off when decryption fails**

If a private note cannot be decrypted, toggling is_private to False must not succeed silently. The system must inform the user that the note body could not be recovered. Now explicit in refined FR-04.

ValidationLayer

- **Title contains only whitespace (spaces or tabs)**

A title of ' ' passes a naive non-empty check. ValidationLayer must strip whitespace before validation. Now explicit in refined FR-01.

- **Tags list contains duplicate entries**

FR-06 does not prohibit duplicate tags. This is allowed for now (preserves user intent) but noted as a known limitation in the planning documentation.

- **Note body exceeds an unexpectedly large size**

No size limit is currently defined. For a local 500-note application this is unlikely to be a practical problem, but a soft warning at a reasonable threshold (e.g., 100 KB) is a candidate for a later sprint.

Search (FR-07)

- **Search is called on an empty note collection**

Must return an empty result set, not an error. The Week 2.2 acceptance criteria covered the no-match case but implicitly assumed at least one note exists. Now explicit in refined FR-07.

- **Search keyword contains special regex characters**

If keyword matching were implemented using regex, characters like '.', '*', '(' could cause unexpected behavior or crashes. Refined FR-07 now specifies plain string matching, removing this risk entirely.

4. How AI Helped and What I Refined

AI was used to pressure-test the existing requirement set, following the critique mode described in Section 1.2 of my Working Agreement. The goal this week was not to have AI draft new requirements but to have it challenge what I already wrote.

What AI helped identify:

I gave AI my Week 1.2 requirement set alongside the user stories and acceptance criteria from Week 2.2 and asked it to find ambiguities, missing edge cases, and weak assumptions. The AI correctly flagged the missing

hardware baseline in NFR-01 and NFR-03, the undefined behavior for toggling is_private off (A-02), and the gap around private notes in search results (A-03). These matched issues I had already noticed during the Sprint Zero planning work but had not formally resolved. The AI also produced a useful first-pass list of storage-layer edge cases - corrupt files, missing directory, filename/UUID mismatch - that were accurate for the architecture.

What I changed or rejected:

The AI flagged "no offline support" as an ambiguity. AstraNotes is a local-only application by design; this is a settled scope decision from the architecture, not an ambiguity. I rejected this.

The AI suggested adding a requirement for note versioning and undo history. This is a new feature, not a refinement of an existing requirement. I rejected it and noted it as a future backlog idea, consistent with the scope discipline rules in my Working Agreement (Section 1.2).

For SPR-01, the AI proposed requiring OS-level keychain integration as the key management solution. This is a reasonable implementation approach, but it is a design decision, not a requirement. I revised the requirement to mandate that the strategy be documented in the ADL before Sprint 1, leaving the specific mechanism open. This distinction between requirements and design was reinforced by the Week 3.1 live examples (Example 3: requirement mixed with design choice).

The AI's edge-case list included "concurrent writes from multiple users." AstraNotes is a single-user local application; concurrent access is not a realistic edge case for this scope. I removed it.

The refinement process confirmed that the Week 1.2 requirements were well-scoped but had meaningful gaps - particularly around privacy edge cases, undefined failure modes, and the distinction between requirements and design decisions. The six ambiguities and eleven edge cases documented above are the ones that would have caused real implementation confusion in Sprint 1 if left unaddressed.